

Relazione - Versione 8: Attention

Introduzione

La Versione 8 sostituisce l'elaborazione ricorrente con gate della Versione 7 con una scaled dot-product attention esplicita.

Restano invariati corpus, vocabolario, finestre di contesto di quattro caratteri, embedding appresi, split training/validation, mini-batch, cross-entropy, differenziazione automatica e campionamento autoregressivo. La ricorrenza viene rimossa: embedding posizionali addestrabili preservano l'ordine e proiezioni query, key e value permettono alle posizioni del contesto di scambiare informazione direttamente.

```
Caratteri
↓
Identificatori interi
↓
Contesto di 4 caratteri
↓
Embedding carattere + posizione
↓
Proiezioni query, key e value
↓
Scaled dot-product attention
↓
Rappresentazione contestuale finale
↓
Pesi di output
↓
Testo generato
```

Obiettivo della Versione 8

Il modello deve essere in grado di:

- riutilizzare lo stesso corpus di 440 caratteri, il vocabolario di 27 caratteri e le finestre di contesto di quattro caratteri della Versione 7;
- riutilizzare embedding addestrabili a 8 dimensioni e lo stesso compito di previsione del carattere successivo;
- aggiungere embedding posizionali addestrabili per distinguere le quattro posizioni senza ricorrenza;
- costruire esplicitamente la scaled dot-product attention tramite matrici di proiezione query, key e value;
- mantenere training mini-batch, validation e generazione autoregressiva riproducibile;
- verificare forme e pesi di attention, gradienti, aggiornamenti, retraining deterministico e generazione.

Architettura

Il flusso completo della Versione 8 è:

```
Corpus: 440 caratteri
↓
Vocabolario: 27 caratteri
↓
436 esempi contesto-target da 4 caratteri
↓
Tensore input intero: (436, 4)
↓
Embedding caratteri: (27, 8) + posizioni: (4, 8)
↓
Rappresentazioni Q / K / V: 4 x 16 per esempio
↓
Matrice di attention: 4 x 4 per esempio
↓
Rappresentazione contestuale finale: 16 valori
↓
Matrice pesi di output: (16, 27)
↓
Split 80 / 20 riproducibile + aggiornamenti mini-batch
↓
Generatore autoregressivo a finestra scorrevole
```

Il contesto contiene ancora quattro caratteri precedenti. Gli embedding dei caratteri e quelli posizionali formano l'input dell'attention, mentre la rappresentazione contestuale della posizione finale predice il carattere successivo.

Finestre di contesto, split e mini-batch

Le finestre scorrevoli creano gli stessi 436 esempi input-target della Versione 7. Ogni esempio contiene quattro identificatori di carattere ordinati; il carattere target si trova fuori dalla finestra di input.

```
esempi training = 348
esempi validation = 88
dimensione batch = 32
aggiornamenti per epoca = 11
```

Solo gli esempi di training vengono mescolati e usati per aggiornare i parametri. Gli esempi di validation misurano la loss senza modificare il modello. Le finestre vicine continuano a sovrapporsi, mantenendo la stessa limitazione intenzionale del piccolo dataset delle versioni precedenti.

Embedding addestrabili e proiezioni di attention

Ogni identificatore di carattere seleziona una riga dalla matrice di embedding addestrabile. Un vettore posizionale addestrabile viene aggiunto in ciascuna delle quattro posizioni del contesto. Le rappresentazioni risultanti vengono proiettate in query, key e value prima di applicare la scaled dot-product attention.

```
Forma tensore input: (436, 4)
Forma matrice embedding caratteri: (27, 8)
Forma matrice embedding posizionali: (4, 8)
Contesto embedded / posizionato: (1, 4, 8)
Forma matrice pesi query: (8, 16)
Forma matrice pesi key: (8, 16)
Forma matrice pesi value: (8, 16)
Query / key / value: (1, 4, 16)
Forma pesi di attention: (1, 4, 4)
Forma output attention: (1, 4, 16)
Forma matrice pesi output: (16, 27)
Forma logits: (1, 27)
Parametri addestrabili: 1064
```

La matrice di embedding dei caratteri contiene 216 valori e quella degli embedding posizionali ne contiene 32. Le matrici di proiezione query, key e value contengono 128 valori ciascuna, mentre la matrice di output ne contiene 432. Il modello ha quindi 1064 parametri addestrabili, rispetto ai 1800 della Versione 7.

```
Q = X W_q, K = X W_k, V = X W_v
scores = Q K^T / sqrt(d_k)
attention = softmax(scores)
H = attention V
```

Loss stabile in TensorFlow

Gli identificatori del contesto vengono mappati negli embedding dei caratteri e combinati con gli embedding posizionali. L'attention produce rappresentazioni contestuali e la posizione finale viene moltiplicata per i pesi di output per produrre i logits del carattere successivo. La sparse softmax cross-entropy misura l'errore di previsione direttamente dai logits e dagli ID target interi.

```
_, _, logits = attention_forward(..., inputs)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

Le loss iniziali di training e validation sono circa 3.2958, coerenti con una previsione quasi uniforme sui 27 caratteri.

Esecuzione controllata e riproducibilità

Il notebook usa la CPU e sopprime i log TensorFlow di basso livello. Seed espliciti controllano inizializzazione, split, shuffle dei mini-batch e campionamento. Le esecuzioni sono deterministiche nello stesso ambiente software.

Differenziazione automatica

Per ogni mini-batch, `tf.GradientTape` registra il forward dell'attention e deriva la loss rispetto alle sei matrici addestrabili:

- `tf.GradientTape` registra embedding, posizione, proiezioni Q/K/V, score, scaling, softmax, value pesati, output e loss;
- `tape.gradient` calcola i gradienti per embedding, tre proiezioni e pesi di output;
- il gradiente sparso degli embedding viene convertito in un tensore denso;
- `assign_sub` applica aggiornamenti di gradient descent a tutte e sei le variabili;
- le loss di training e validation vengono registrate a ogni epoca.
 with `tf.GradientTape()` as `tape`:
 `batch_loss = calculate_loss(...)`
 `gradients = tape.gradient(batch_loss, trainable_variables)`

Configurazione training:

```
Learning rate: 1.0  
Dimensione batch: 32  
Epoche: 100  
Seed: 42
```

Risultato del training

```
Loss training iniziale: 3.295835  
Loss training finale: 2.735393  
Loss validation iniziale: 3.295837  
Loss validation finale: 2.927145  
Migliore loss validation: 2.884859  
Migliore epoca validation: 74
```

Il modello cambia poco nelle prime 20 epoche, poi la loss diminuisce più chiaramente. Con learning rate fisso il training resta oscillatorio, ma la loss di training cala sensibilmente entro l'epoca 100.

La validation è migliore all'epoca 74. In seguito la loss di training cala mentre quella di validation cresce leggermente, suggerendo lieve overfitting. La generazione usa i parametri finali dell'epoca 100.

Controlli su attention e parametri

I test verificano gradienti finiti per tutte e sei le matrici di parametri addestrabili, forme dei gradienti corrispondenti, output contestuali finiti, pesi di attention normalizzati e che ogni matrice di parametri cambi effettivamente durante il training.

```
0 <= attention_weights <= 1
sum(attention_weights, axis=-1) = 1
assert all(gradient is not None for gradient in gradients)
assert np.max(abs(initial - trained)) > 0.0
```

Probabilità apprese

Per il contesto mode, le probabilità apprese più alte per il carattere successivo sono:

```
spazio: 0.0897   e: 0.0879   a: 0.0698
n: 0.0610       s: 0.0588   t: 0.0581
```

Ispezione dei pesi di attention

L'attention espone una matrice 4 x 4 le cui righe sommano a 1. Per mode, la riga finale usata dal layer di output è:

```
m: 0.2422   o: 0.2643   d: 0.1634   e: 0.3301
somma riga finale: 0.99999994
```

Generazione neurale del testo

Per ogni previsione, la finestra corrente di quattro caratteri viene elaborata dall'attention, la rappresentazione contestuale finale produce i logits e un generatore NumPy locale campiona il carattere successivo prima che la finestra avanzi.

```
mode -> carattere campionato
ode? -> carattere successivo campionato
...
```

Con seed 42, cinque campioni dal contesto mode sono r, i, t, p e virgola. La dimostrazione genera poi 300 caratteri riproducibili.

Controlli inclusi

Le asserzioni verificano costruzione del contesto, tipi di tensori e variabili, forme dei parametri, intervalli e somme delle righe dell'attention, split dei dati, loss, probabilità, gradienti, modifiche dei parametri, retraining deterministico e generazione riproducibile.

Un'esecuzione completa dall'inizio alla fine termina con il messaggio All checks passed.

Cosa insegna la Versione 8

La progressione centrale è:

contesto -> ID -> embedding carattere + posizione -> Q / K / V
-> attention scalata -> rappresentazione contestuale -> logits -> loss
-> gradienti -> aggiorna -> scorri -> campiona

La Versione 8 sostituisce il flusso ricorrente con uno scambio diretto basato sull'attention: query e key confrontano le posizioni, mentre i value trasportano l'informazione combinata.

Interpretazione

Gli embedding posizionali preservano esplicitamente l'ordine e le stesse proiezioni query, key e value sono riutilizzate in ogni posizione. Gli output contestuali sono calcolati per tutte le posizioni, mentre la previsione usa solo quella finale.

Il modello ha 1064 parametri contro i 1800 della Versione 7. La migliore loss di validation è 2.8849 all'epoca 74, leggermente sotto il 2.8995 della Versione 7; su questo piccolo corpus il confronto è solo illustrativo.

Limitazioni

- il contesto mantiene una lunghezza fissa di quattro caratteri e il corpus resta molto piccolo;
- finestre di contesto sovrapposte possono comparire tra lo split casuale di training e validation;
- il modello non usa bias né causal mask; il target resta fuori dal contesto di input;
- solo la rappresentazione finale contribuisce direttamente alla loss di previsione;
- c'è un solo calcolo di attention e nessuna struttura residual, normalization, feed-forward o multi-head;
- il training usa gradient descent mini-batch a learning rate fisso, senza early stopping o checkpoint;
- il testo generato cattura pattern locali ma non ha ancora coerenza a lungo termine.

Conclusione

La Versione 8 sostituisce la ricorrenza con gate con scaled dot-product attention esplicita. Gli embedding posizionali preservano l'ordine, i confronti query-key producono pesi normalizzati e i value pesati formano rappresentazioni contestuali.

L'implementazione resta ispezionabile. La Versione 9 aggiunge causal self-attention, feed-forward, connessioni residual e normalizzazione.